

UNIVERSIDAD AUTÓNOMA DE SAN LUÍS POTOSÍ



FACULTY OF ENGINEERING

Project Report

Detecting Gravitational Lenses Using Convolutional Neural Networks

Author:

Macías Macías Francisco Uriel

Advisor:

Dr. Cuevas Tello Juan Carlos

Machine Learning Course

December, 2025

Abstract

Detecting gravitational lenses in large astronomical surveys is a challenging task due to the subtle morphological features present in small and noisy galaxy images. In this work, a convolutional neural network (CNN) was developed to automatically classify strong gravitational lensing systems using 41×41 single-band (r -band) FITS images from both real Hyper Suprime-Cam (HSC) observations and simulated data. The dataset was reorganized into a binary classification task, combining real and simulated examples into two categories: lenses and non-lenses. After preprocessing, normalization, and balanced stratified splitting, the model was trained using regularization techniques such as batch normalization, dropout, weight decay, and early stopping to ensure stable convergence and reduce overfitting.

The proposed CNN achieved strong performance, obtaining a test accuracy of 0.909 and a test AUC of 0.965. The confusion matrix and classification report show balanced precision and recall for both classes, indicating reliable generalization across real and simulated domains. These results demonstrate that compact CNN architectures can effectively extract lensing features such as arcs and partial Einstein rings, even from low-resolution images.

This work highlights the potential of deep learning as a scalable tool for automated lens discovery in next-generation surveys such as LSST or Euclid, where manual inspection is infeasible. Future extensions include multi-band integration, advanced data augmentation, explainability techniques, and adapting the model into a full candidate-ranking pipeline for large-scale astrophysical analyses.

Contents

1	Introduction	2
1.1	Background	2
1.2	Motivation	2
2	State of Art	3
2.1	Overview	3
2.2	Recent Datasets and Challenges	3
3	Methodology	4
3.1	Layers of a Convolutional Neural Network	4
3.2	Methodology for This Project	5
4	Datasets	6
4.1	Dataset Description	6
4.2	Image Preparation	7
4.3	Importance of Outputs	9
5	AI Methods and Algorithms	10
5.1	Overview of the Algorithms	10
5.2	Network Architecture	11
5.3	Training Configuration	14
6	Results	16
6.1	Training and Validation Performance	16
6.2	Test Set Evaluation	17
6.3	Confusion Matrix and Classification Report	17
6.4	Discussion	18
7	Conclusions	19
7.1	Future Work	19
	References	21

Introduction

1.1 Background

Gravitational lenses occur when a massive object deforms space and time, causing light to bend, distort, and magnify as it passes around the object. The study and detection of gravitational lenses is an area of great interest due to its applications in the search for dark matter, exoplanets, and in understanding the expansion of the universe.

Galaxies can act as large-scale gravitational lenses, but there are also microlenses within galaxies, which are particularly useful for the detection of MACHOs. By mapping their positions, it is possible to delimit regions that contain large amounts of dark matter (Rubio Viana, 2023).

1.2 Motivation

Moreover, the study of gravitational lenses has allowed astronomers to map the large-scale distribution of galaxies in the universe. Observatories such as LSST provide a census of millions of galaxies solely through gravitational lensing. Thanks to this phenomenon, searches can also be conducted for the discovery of distant radio galaxies, helping to understand how some of the earliest galaxies in the universe formed, grew, and evolved (Gravitational Lens Effect — Center for Astrophysics — Harvard & Smithsonian, 2022).

Determining the masses of astronomical objects represents a challenge for observational techniques, since mass is not a directly observable property and is generally inferred from the motion of components or their luminosity. This requires making certain assumptions, which may introduce biases into the measurements (Johana González, 2017).

Gravitational lenses are phenomena identified through the analysis of astronomical images, which are often examined manually, a process that requires significant time and effort. Evidently, this method is prone to errors due to the vast amount of data astronomers must process, as well as the difficulty of distinguishing gravitational lenses from other similar structures, which significantly increases the margin of error.

In this context, this project explores the use of convolutional neural networks to automatically distinguish between images that contain strong gravitational lenses and those that do not.

State of Art

2.1 Overview

With the growing use of artificial intelligence in various fields of science, the study of gravitational lenses has gained great relevance in astronomy, especially for research on dark matter, the expansion of the universe, and the detection of distant objects. In recent years, this field has undergone a revolution thanks to the integration of artificial intelligence algorithms that allow the automated detection of gravitational lenses through convolutional neural networks (CNNs) and deep learning models.

2.2 Recent Datasets and Challenges

An example of this is the Kavli Institute for Particle Astrophysics and Cosmology (KIPAC), a joint institution of SLAC and Stanford, whose researchers demonstrated that neural networks can transform the way astrophysicists analyze complex data. The team used neural networks to examine images of strong gravitational lenses, in which the image of a distant galaxy is multiplied and distorted into rings due to the gravity of a massive object, such as a galaxy cluster. With this technique, the researchers were able to carry out the same analysis in just seconds—a process that previously required tedious manual comparison of images. For this purpose, they used approximately half a million simulated images of gravitational lenses, in a training process that took about one day. This occurred in 2017 (Artificial Intelligence Analyzes Gravitational Lenses 10 Million Times Faster — SLAC National Accelerator Laboratory, 2017).

In 2022, a group of international researchers employed an artificial intelligence system to identify 5,000 possible gravitational lenses, marking the first large-scale detection of these phenomena. The data were obtained using instruments from the Keck Observatory in Hawaii and the Very Large Telescope in Chile, achieving an effectiveness of 88% certainty (“The AGEL Survey: Spectroscopic Confirmation of Strong Gravitational Lenses in the DES and DECaLS Fields Selected Using Convolutional Neural Networks,” 2022).

Likewise, studies such as A Comparative Study of Convolutional Neural Networks for the Detection of Strong Gravitational Lensing have demonstrated the effectiveness of different neural network architectures for detecting these phenomena, reaching levels of accuracy considered high (Magro et al., 2021).

Thanks to these advances, the implementation of artificial intelligence in astronomy has opened new opportunities to improve and automate image recognition, reducing analysis time and enabling new discoveries. In this context, the present project aims to design artificial intelligence algorithms capable of achieving an acceptable level of accuracy in the detection of gravitational lens images.

Methodology

As can be seen in the state of the art, convolutional neural networks (CNNs) have greatly helped solve this type of problem. These are machine learning models that use three-dimensional data to perform tasks such as image classification or object recognition. They achieve this because they are composed of nodes that have input layers, one or more hidden layers, and an output layer. Each node is connected to another with an associated weight and threshold. If the output value exceeds this threshold, the node is activated and sends data to the next layer of the network; otherwise, no data is passed (IBM, n.d.).

3.1 Layers of a Convolutional Neural Network

A convolutional neural network is divided into several layers:

1. **Convolutional layer:** Features, both simple and complex, are assigned throughout the image. Each time a feature is detected, it is recorded in a feature map.
2. **Activation layer (ReLU):** After the initial scan, the feature maps are reviewed; strong signals are kept and weak ones are discarded. This prevents noise from interfering with the process.
3. **Pooling layer:** Instead of analyzing every detail, this layer summarizes the findings of small patches of the feature map. Pooling layers retain only the strongest signals, reducing the overall data size.
4. **Fully connected layer:** After several rounds of scanning, the final results reach this layer, which analyzes the combination of high-level features and makes the final decision. This connects all findings to extract a conclusion (¿Qué Es Una Red Neuronal Convolutacional?, n.d.).

It is important to highlight that CNNs can have dozens or even hundreds of these types of layers, which differentiates them from standard neural networks. They also have shared weights and bias values, meaning that all hidden neurons detect the same features (¿Qué Son Las Redes Neuronales Convolucionales?, n.d.).

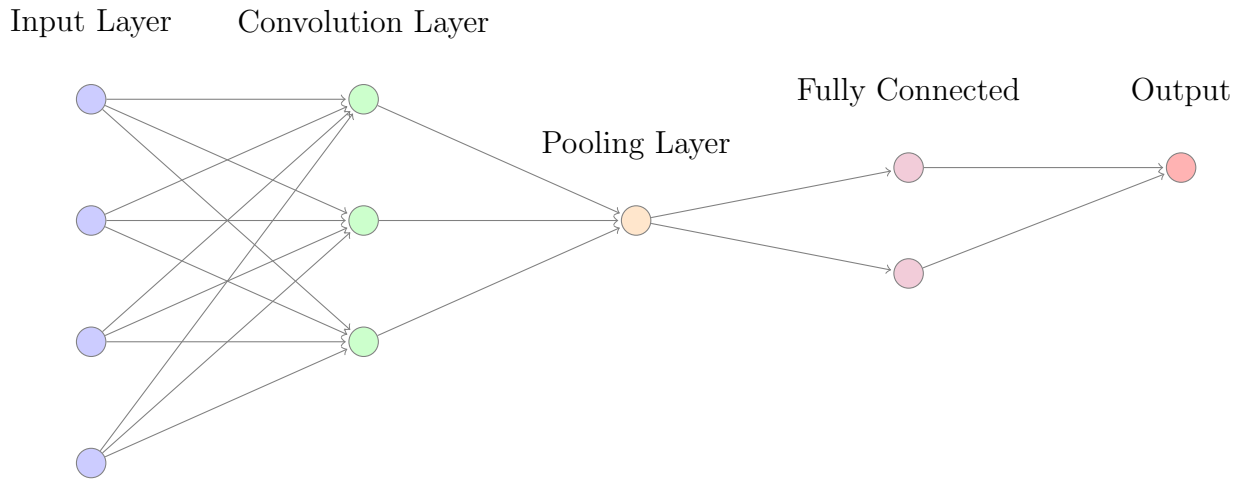


Figure 3.1: Graph representation of a Convolutional Neural Network (CNN).

3.2 Methodology for This Project

Since the images used for this project contain a large amount of data, this model was chosen. The stages for training and executing this model are as follows:

- **Data splitting:** Dividing the dataset into training, validation, and test sets, while defining an appropriate number of epochs to achieve optimal results.
- **Optimization and evaluation:** Using a loss function to optimize the model, calculating the binary cross-entropy loss, and generating probabilistic data (confusion matrix) to determine the model's effectiveness and accuracy.
- **Tools and implementation:** Programming languages and libraries such as Python and Tensorflow/Keras will be used, all organized and executed using a Jupyter Notebook.

Datasets

The dataset used for this project was taken from the Static Strong Lenses Finding Challenge Dataset (LSST).

This challenge is part of a series on strong gravitational lenses organized by the LSST Dark Energy Science Collaboration (DESC) and the Strong Lensing Science Collaboration (SLSC) to develop and test machine learning algorithms designed to identify these phenomena, as prepared for the Vera C. Rubin Observatory.

The main purpose of this challenge is to realistically simulate data for training and testing algorithms aimed at detecting strong gravitational lenses on a galactic scale. The simulations use data extracted from sources such as the Hubble Space Telescope (HST) and the Hyper Suprime-Cam (HSC PDR3) (Strong Lens Challenge Data Release, 2025).

4.1 Dataset Description

The dataset consists of a total of one million astronomical images in **FITS (Flexible Image Transport System)** format. This format is very common in astronomy, mainly for storing digital images that contain data in matrix form.

The images are grouped into four main categories:

- Gravitational lenses captured by the Hyper Suprime-Cam (HSC)
- Non-lens objects captured by HSC
- Simulated gravitational lenses generated by SIMCT
- Simulated non-lens objects generated by SIMCT

FITS images contain not only the image itself but also metadata related to astronomical observations, including telescope information, capture date, coordinates of the object in the sky, and other important parameters (FORMATO FITS, n.d.). Using the **Astropy** library, it is possible to load FITS files and extract pixel matrices, which can then be manipulated with other Python libraries.

These images are subdivided by the band (filter) in which they were captured. There are five bands: g, r, i, z, and y, producing five channels per object. This results in a total of 50,000 examples per category: real lenses, real non-lenses, simulated lenses, and simulated non-lenses.

4.2 Image Preparation

For this experiment, and due to the computational limitations of the equipment used, the total number of images was reduced to only 10,000 per class, resulting in a total of 20,000 images. Additionally, only the r-band was used. This band lies within the grayscale spectrum of the pixelated images, representing the light intensity over a specific wavelength range — each pixel shows how much light was captured by the telescope. The r-band, with red as its dominant color, can detect visible mid-range light, allowing the tracing of intermediate regions of the spectrum, which is very common in morphological studies of galaxies.

By working with this band, we achieve a better signal-to-noise ratio while maintaining good angular resolution, along with advantages such as the preservation of recognizable patterns in the images, which have a resolution of 41×41 pixels.

This pixel format represents about 6–7 arcseconds squared of the sky — a small but sufficient crop to capture the structure of a lens. To the human eye, objects such as galaxies or lens arcs appear blurry at this resolution, but this is not an issue for the CNN; what matters are the intensity and symmetry patterns. Likewise, the values of this 41×41 matrix are normalized to a range between 0 and 1.

This normalization is necessary because the raw image data contain intensity values that can vary greatly across different ranges. Without normalization, training a neural network would be slower, activation functions would saturate, and optimization would become unstable. By normalizing, we obtain more comparable contrast and scale values, improving the model’s learning performance.

Likewise, each of the images is stored in a NumPy array. Each image file has a name that includes the gravitational lens ID and a keyword indicating which band it belongs to. With this information, we can build two NumPy arrays: the first one contains all the images (X), and the second one contains all the corresponding labels.

In this way, by using the same index in both arrays, we can access an image and determine which of two classes it belongs to. These classes are 0 and 1, representing: gravitational lenses and non-gravitational lenses, respectively.

Thus, both arrays have the following dimensions:

X.shape=(20000, 41, 41, 1), y.shape=(20000,)

It is also important to define the percentage of data that will be used for training, validation, and testing. For this, we can perform the following split:

Listing 4.1: Data split for training and testing

```
# Split 80/20
SEED = 42
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=SEED,
    stratify=y
)
```

This defines 80% of the data for training and 20% for testing. Later, during the training process, 15% of the training data will be used for validation.

Shape X_train: (16000, 41, 41, 1) — y_train: (16000,)

Shape X_test : (4000, 41, 41, 1) — y_test : (4000,)

Class in train: {1: 8000, 0: 8000}

Class in test : {1: 2000, 0: 2000}

4.3 Importance of Outputs

The labeled images are fundamental for both training and evaluating the convolutional neural network. In this project, four distinct categories were defined: lenses and non-lenses. Each image was assigned an integer label 0 or 1, corresponding to its respective class. These labels guide the learning process, allowing the model to associate pixel-level patterns with physical phenomena such as gravitational lensing or the absence thereof.

Having a balanced number of examples in each of the four categories ensures that the model is not biased toward any specific class. This balance improves the fairness and stability of training, leading to a network that can generalize more effectively to unseen data. Furthermore, the diversity among the classes — spanning both real and simulated images — provides the model with richer visual cues and a broader representation of the astrophysical variations present in strong lensing systems.

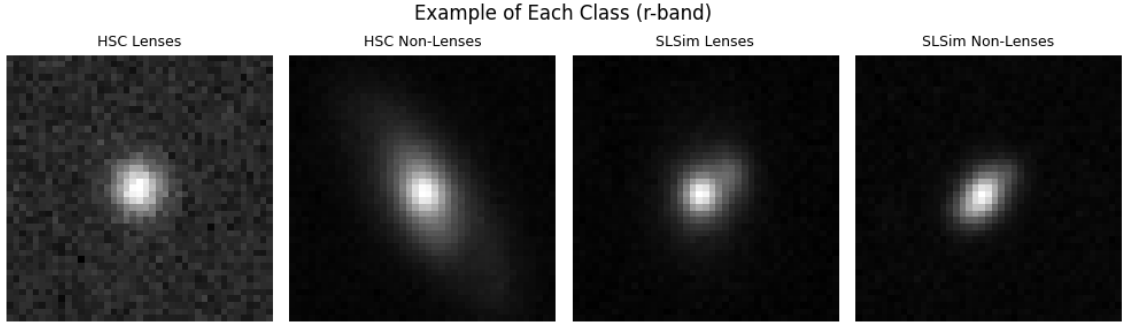


Figure 4.1: Examples of the four categories used in this work: HSC lenses, HSC non-lenses, SLSim lenses, and SLSim non-lenses.

The model’s outputs can be interpreted as the likelihood of an astronomical object exhibiting features consistent with gravitational lensing. High predicted probabilities for lens classes indicate the presence of circular or arc-like structures typical of strong gravitational lenses, while high probabilities for non-lens classes correspond to more regular galaxy morphologies.

AI Methods and Algorithms

5.1 Overview of the Algorithms

A Convolutional Neural Network (CNN) is a type of artificial neural network specialized in processing data with a grid-like structure, such as images. Its main function is to identify and extract patterns through convolution, activation, and pooling layers, allowing it to perform complex visual tasks such as recognition and object detection. CNNs are a form of deep learning that “learns to see” by analyzing and understanding data in a way that emulates human visual perception (MathWorks, n.d.).

This architecture is inspired by the visual cortex of the human brain and operates through a series of interconnected layers that can be divided into three main types:

- **Convolutional layers**
- **ReLU activation layers**
- **Pooling layers**

CNNs are most commonly used in computer vision applications such as image classification and object detection, but they are also effective in other domains, including medical imaging for detecting cellular anomalies, audio signal processing, keyword recognition, and natural language processing (Arm Ltd., n.d.).

Given their ability to automatically extract visual patterns, CNNs are particularly well-suited for this project, which involves classifying images into two categories of gravitational lens systems. Since the input data consist of pixel matrices with varying intensity levels, the model can learn to detect key features such as Einstein rings and the arc-like structures typical of gravitational lenses. The implementation of this model was carried out using the **TensorFlow** and **Keras** frameworks.

5.2 Network Architecture

The proposed convolutional neural network was designed to classify astronomical images into two categories corresponding to lenses and non-lenses. The model receives grayscale inputs with a spatial resolution of 41×41 pixels and a single channel, matching the r -band images extracted from the dataset. Its architecture follows a hierarchical structure composed of three convolutional blocks, each aimed at progressively capturing spatial patterns and higher-level features.

Each block consists of two consecutive **Conv2D** layers with 3×3 kernels, followed by **Batch Normalization** and **ReLU** activation. The first block starts with 32 filters, the second with 64, and the third with 128, doubling the number of filters at each stage to increase representational capacity. A **MaxPooling2D** layer with a 2×2 window is applied after each block to reduce spatial dimensionality and computational cost while preserving the most salient features.

After the convolutional stage, the feature maps are summarized using a **Global Average Pooling (GAP)** layer, which replaces traditional flattening and reduces the risk of overfitting by averaging spatial activations. This is followed by a **Dropout** layer (rate = 0.5) to further improve regularization and prevent the network from relying excessively on specific neuron activations.

The dense stage of the model consists of a fully connected layer with 64 units and ReLU activation, which integrates the extracted features, and a final **Dense(1)** output layer with a **Sigmoid** activation that produces a probability distribution across the two classes.

Table 5.1 summarizes the layer structure and parameter count of the model.

Table 5.1: Summary of the CNN architecture used for gravitational lens classification.

Layer (Type)	Output Shape	Parameters
Conv2D (32 filters, 3×3)	(41, 41, 32)	288
BatchNormalization + ReLU	(41, 41, 32)	128
Conv2D (32 filters, 3×3)	(41, 41, 32)	9,216
BatchNormalization + ReLU	(41, 41, 32)	128
MaxPooling2D (2×2)	(20, 20, 32)	0
Conv2D (64 filters, 3×3)	(20, 20, 64)	18,432
BatchNormalization + ReLU	(20, 20, 64)	256
Conv2D (64 filters, 3×3)	(20, 20, 64)	36,864
BatchNormalization + ReLU	(20, 20, 64)	256
MaxPooling2D (2×2)	(10, 10, 64)	0
Conv2D (128 filters, 3×3)	(10, 10, 128)	73,728
BatchNormalization + ReLU	(10, 10, 128)	512
Conv2D (128 filters, 3×3)	(10, 10, 128)	147,456
BatchNormalization + ReLU	(10, 10, 128)	512
GlobalAveragePooling2D	(128)	0
Dropout (rate = 0.5)	(128)	0
Dense (64, ReLU)	(64)	8,256
Dense (1, Sigmoid)	(1)	—

This architecture strikes a balance between depth and computational efficiency,

allowing the model to learn relevant spatial features even with relatively small 41×41 inputs. The combination of convolutional feature extraction, normalization, and global pooling contributes to stable convergence and high classification accuracy on both real and simulated data.

Listing 5.1: CNN architecture

```
import tensorflow as tf

modelo = tf.keras.Sequential([
    # Input
    tf.keras.layers.Input(shape=(41, 41, 1)),

    # Bloque 1: 32 filtros
    tf.keras.layers.Conv2D(32, (3, 3), padding="same",
                           kernel_regularizer=tf.keras.
                           regularizers.l2(1e-4),
                           use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),

    tf.keras.layers.Conv2D(32, (3, 3), padding="same",
                           kernel_regularizer=tf.keras.
                           regularizers.l2(1e-4),
                           use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),

    tf.keras.layers.MaxPooling2D((2, 2)),    # 41x41 -> ~20x20

    # Bloque 2: 64 filtros
    tf.keras.layers.Conv2D(64, (3, 3), padding="same",
                           kernel_regularizer=tf.keras.
                           regularizers.l2(1e-4),
                           use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),

    tf.keras.layers.Conv2D(64, (3, 3), padding="same",
                           kernel_regularizer=tf.keras.
                           regularizers.l2(1e-4),
                           use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),

    tf.keras.layers.MaxPooling2D((2, 2)),    # ~20x20 -> ~10
    x10

    # Bloque 3: 128 filtros
    tf.keras.layers.Conv2D(128, (3, 3), padding="same",
                           kernel_regularizer=tf.keras.
                           regularizers.l2(1e-4),
```

```

        use_bias=False),
tf.keras.layers.BatchNormalization(),
tf.keras.layers.ReLU(),

tf.keras.layers.Conv2D(128, (3, 3), padding="same",
                        kernel_regularizer=tf.keras.
                        regularizers.l2(1e-4),
                        use_bias=False),
tf.keras.layers.BatchNormalization(),
tf.keras.layers.ReLU(),

# Head de clasificaci n
tf.keras.layers.GlobalAveragePooling2D(),    # evita
    Flatten con muchos par metros
tf.keras.layers.Dropout(0.5),
tf.keras.layers.Dense(64, activation="relu"),
tf.keras.layers.Dense(1, activation="sigmoid") # 2
    clases (clasificaci n binaria)
])

```

5.3 Training Configuration

The model was trained using the **Adam** optimizer with an initial learning rate of 1×10^{-3} . This optimizer was selected for its adaptive learning rate capabilities, which provide stable and efficient convergence in deep learning models. The loss function employed was the **Binary Crossentropy**, appropriate for binary classification tasks. In addition to accuracy, the Area Under the ROC Curve (AUC) metric was used to better quantify the model's ability to distinguish between lenses and non-lenses.

Training was carried out for a maximum of 50 epochs with a batch size of 64, using 15% of the training data as a validation split. The model was trained on a local workstation equipped with an Intel Core i7 (8th generation) processor, 8 GB of RAM, and an SSD drive. Due to the hardware constraints, no dedicated GPU acceleration was used, which influenced training time but not the final model performance.

To enhance the stability and robustness of the training process, several **callbacks** were integrated:

- **ModelCheckpoint** automatically saved the best model weights based on validation accuracy, preventing the loss of optimal states during training interruptions.
- **EarlyStopping** monitored the validation loss and halted training when no improvement was observed for six consecutive epochs, restoring the best-performing weights.
- **ReduceLROnPlateau** dynamically reduced the learning rate by a factor of 0.5 when the validation loss plateaued for three epochs, allowing finer convergence.
- **CSVLogger** stored detailed epoch-by-epoch training logs for further analysis and reproducibility.

This configuration ensured an efficient and reliable training process, minimizing overfitting while maintaining high validation performance. The model reached optimal performance around epoch 22, after which validation accuracy stabilized, confirming effective generalization across unseen data.


```

from keras.callbacks import ModelCheckpoint, EarlyStopping,
    ReduceLROnPlateau, CSVLogger

loss = tf.keras.losses.BinaryCrossentropy()

modelo.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3), # Optimizador
        de los pesos
    loss=loss, # Funcion de perdida, cuan lejos estan las
        predicciones
    metrics=["accuracy", tf.keras.metrics.AUC()]
)

callbacks = [
    ModelCheckpoint(
        filepath="best.keras",
        monitor="val_accuracy",
        mode="max",
        save_best_only=True,
        verbose=1
    ),
    EarlyStopping(
        monitor="val_loss",
        patience=6,
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=3,
        verbose=1
    ),
    CSVLogger("train_log.csv", append=False)
]

hist = modelo.fit(
    X_train, y_train,
    epochs=50, # Numero de epocas maximo
    batch_size=64,
    validation_split=0.15, # usar 15% del train
        como validaci n
    callbacks=callbacks,
    verbose=1
)

```

Results

This section presents the performance of the convolutional neural network after training on the binary classification task of distinguishing gravitational lenses from non-lenses. The model was trained for a maximum of 50 epochs using early stopping and learning rate reduction callbacks. The best-performing model was automatically saved based on validation accuracy.

6.1 Training and Validation Performance

Figure 6.1 shows the accuracy curves for both training and validation sets across epochs. The network exhibits a smooth and monotonic increase in training accuracy, while validation accuracy stabilizes after approximately the 12th epoch. The highest validation accuracy was achieved at epoch 18, with a value of 0.923. At that point, the model parameters were saved as the best configuration.

During epoch 18, the model achieved the following performance:

- Training accuracy: 0.9556
- Training AUC: 0.9920
- Training loss: 0.1426
- Validation accuracy: 0.9229 (best epoch)

The relatively small gap between training and validation accuracy indicates that the network generalized well and shows minimal signs of overfitting.

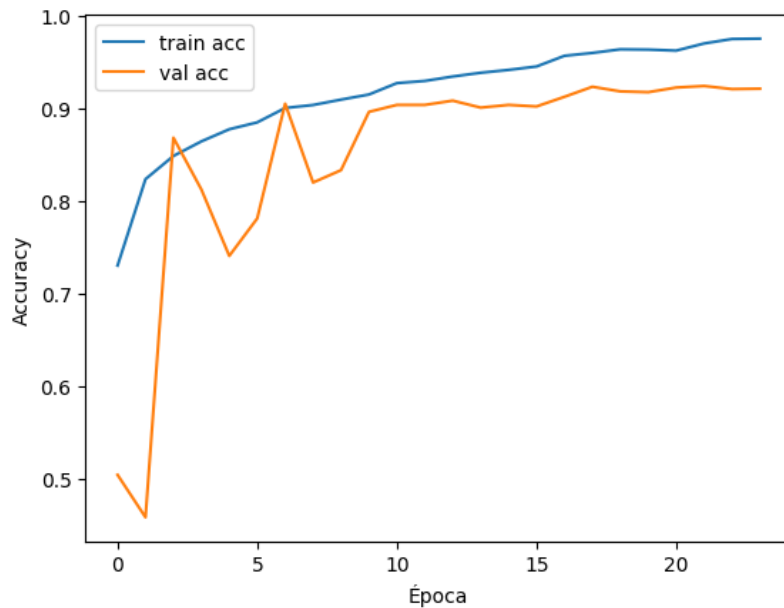


Figure 6.1: Training and validation accuracy across epochs.

6.2 Test Set Evaluation

The best saved model was evaluated on the test set consisting of 4,000 previously unseen images (2,000 lenses and 2,000 non-lenses). The following results were obtained:

- **Test accuracy:** 0.9095
- **Test AUC:** 0.9654
- **Test loss:** 0.3116

The high AUC score indicates that the model has strong discriminative power and is highly effective at separating the two classes across different classification thresholds. The test accuracy above 90% demonstrates robust generalization to unseen data, even though the classifier was trained on only a subset of the original dataset.

6.3 Confusion Matrix and Classification Report

The confusion matrix in Figure 6.2 summarizes the distribution of correct and incorrect classifications. The model correctly identifies 1818 out of 2000 lens images and 1820 out of 2000 non-lens images.

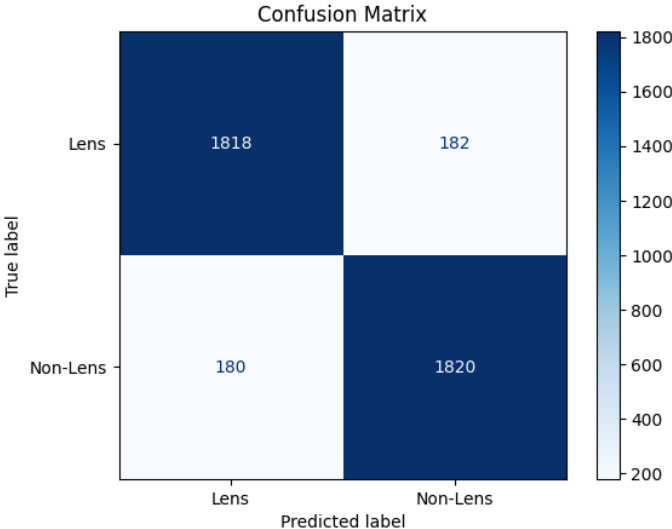


Figure 6.2: Confusion matrix for the binary classification task.

Table 6.1 presents the precision, recall, and F1-score for each class:

The results show excellent balance between the two classes, with nearly identical performance metrics for lenses and non-lenses. The symmetric confusion matrix and the matching precision/recall scores confirm that the model does not favor one class over the other. This is particularly important in astrophysical applications, where class imbalance or model bias can severely compromise scientific reliability.

Table 6.1: Classification report on the test set.

Class	Precision	Recall	F1-score	Support
Lenses	0.910	0.909	0.909	2000
Non-lenses	0.909	0.910	0.910	2000
Accuracy	0.909			
Macro avg	0.910	0.909	0.909	4000
Weighted avg	0.910	0.909	0.909	4000

6.4 Discussion

The model demonstrates strong performance despite being trained exclusively on single-band (r -band) 41×41 images and without GPU acceleration. Achieving over 90% accuracy and an AUC of 0.965 indicates that the CNN successfully learned meaningful morphological patterns associated with gravitational lenses, such as arc-like distortions and Einstein ring structures.

These results validate the capability of convolutional neural networks to detect gravitational lenses using compact, low-resolution astronomical images. Further improvements could be obtained by incorporating multi-band data, increasing the number of training samples, or applying domain-specific augmentation strategies.

Conclusions

In this project, a convolutional neural network was developed and trained to distinguish gravitational lenses from non-lensing astronomical objects using single-band (r -band) 41×41 images. The model achieved strong performance across multiple evaluation metrics, including a test accuracy of 0.909, a test AUC of 0.965, and balanced precision–recall values for both classes. These results demonstrate that even with compact, low-resolution input data, a carefully designed CNN can successfully learn the morphological features associated with strong lensing phenomena, such as arcs, distortions, and partial Einstein rings.

The training process showed stable convergence, with minimal overfitting thanks to the use of regularization techniques such as batch normalization, dropout, weight decay, and early stopping. The confusion matrix and classification report further confirm that the model performs consistently across both categories, indicating that it is capable of generalizing to previously unseen data without significant bias.

Overall, the project validates the effectiveness of deep learning—specifically convolutional architectures—as a tool for automated lens detection in large astrophysical surveys. This methodology has the potential to greatly accelerate the identification of strong lensing candidates in next-generation surveys such as LSST or Euclid, where manual inspection of millions of images is impractical.

7.1 Future Work

Although the results obtained in this work are promising, several extensions could further improve the performance and scientific applicability of the model:

- **Multi-band integration.** The current model uses only the r -band. Incorporating multi-band FITS data (e.g., g , i , z , y) could provide richer color and structural information, enabling the network to better differentiate lensing features from non-lensing galaxies.
- **Data augmentation and domain adaptation.** Additional augmentation strategies targeted to astronomical images—such as realistic PSF variations, noise injection, or small affine distortions—could improve robustness. Domain adaptation techniques may also help reduce the gap between simulated and real lenses.
- **Larger architectures and transfer learning.** Experimenting with deeper CNNs, residual networks (ResNet), or pre-trained models adapted to astronomical domains could lead to higher accuracy. Transfer learning from general-purpose vision models (e.g., EfficientNet) is another promising direction.
- **Explainability and visualization.** Applying techniques such as Grad-CAM or feature attribution methods would help identify which regions of the image contribute most to the classification. This would be useful both for scientific validation and for building trust in automated lens finding pipelines.

- **Automated candidate ranking for surveys.** A real-world extension of this work would incorporate the model into a larger system that prioritizes lens candidates in large-scale surveys, enabling astronomers to inspect the most promising detections efficiently.
- **Expanding to regression or multi-class tasks.** Instead of binary detection, future work could explore predicting lensing strength, arc curvature, or other morphological indicators, opening the door to more detailed physical analysis.

In summary, while the present model already achieves competitive performance, the outlined future steps offer clear opportunities for improvement and further scientific exploration. These enhancements would make the system more accurate, interpretable, and capable of supporting the demands of next-generation astronomical surveys.

References

- Rubio Viana, M. (2023). *Explorando nuevos modelos de materia oscura en el sistema solar a traves de simulaciones de N cuerpos: Trabajo de fin de grado en Física*. <https://zaguan.unizar.es/record/149329/files/TAZ-TFG-2024-4418.pdf>
- Efecto de lente gravitacional — Centro de Astrofísica — Universidad de Harvard y Smithsonian. (2022, October 5). https://www-cfa-harvard-edu.translate.google.com/research/topic/gravitationalensing?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=sge
- Johana Gonzalez, E. (2017). *Observacion y análisis de sistemas astrofísicos usando fenómenos de lentes gravitacionales*. <https://www.famaf.unc.edu.ar/documents/1010/DAst92.pdf>
- Artificial intelligence analyzes gravitational lenses 10 million times faster — SLAC National Accelerator Laboratory. (2017, August 30). SLAC National Accelerator Laboratory. <https://www6.slac.stanford.edu/news/2017-08-30-artificial->
- The AGEL Survey: Spectroscopic confirmation of strong gravitational lenses in the DES and DECALS fields selected using convolutional neural networks. (2022). *Astronomical Journal*. <https://iopscience.iop.org/article/10.3847/1538-3881/ac7da2/pdf>
- Magro, D., Adami, K. Z., DeMarco, A., Riggi, S., & Sciacca, E. (2021). *A comparative study of convolutional neural networks for the detection of strong gravitational lensing*. *Monthly Notices of the Royal Astronomical Society*, 505(4), 6155–6165. <https://doi.org/10.1093/mnras/stab1635>
- Ibm. (n.d.). *Redes neuronales convolucionales*. Recuperado el 16 de septiembre de 2025, de <https://www.ibm.com/mx-es/think/topics/convolutional-neural-networks?text=La%20capa%20convolucional%20es%20la%20se%20identifica%20el%20objeto%20previsto>.
- ¿Qué es una red neuronal convolucional? (n.d.). *Google Cloud*. <https://cloud.google.com/discover/what-are-convolutional-neural-networks?hl=es>
- ¿Qué son las redes neuronales convolucionales? (n.d.). *MATLAB & Simulink*. <https://la.mathworks.com/discovery/convolutional-neural-network.html>
- FORMATO FITS. (n.d.). <https://www.inaoep.mx/~moises/Astrofisica/fits.html>
- Strong lens challenge data release. (2025). *En GOV.BR*. Recuperado el 4 de noviembre de 2025, de https://andromeda.cbpf.br/public/Strong_Lens_Challenge_Data_Release.pdf

- Arm Ltd. (n.d.). *What is a convolutional neural network (CNN)?* Arm — The Architecture for the Digital World. Recuperado de <https://www.arm.com/glossary/convolutional-neural-network>
- MathWorks. (n.d.). *¿Qué son las redes neuronales convolucionales?* MATLAB & Simulink. Recuperado de <https://es.mathworks.com/discovery/convolutional-neural-network.html>